Individual Assignment: Compiler Extensions

IC526 Advanced Compilers
Fall 2026

Extend your RustY compiler with features of your choice. Implement your extension, give a proposal or progress presentation, and submit a final report explaining its design, correctness, and evaluation.

1. Your Task

Work individually. Choose a feature or a coherent set of related features. **A focused project that can be completed in two to three weeks is sufficient.** Define a core goal and, if useful, optional stretch goals.

Your project must:

- Specify the supported inputs, intended behavior or analysis property, and completion criteria.
- Include your own compiler implementation contribution.
- Provide tests and evidence supporting your correctness claims.

You may build on your compiler from earlier assignments and use the provided infrastructure. Clearly distinguish earlier work, reused components, and your new contribution. A limited, carefully specified subset is acceptable.

2. Project Ideas

The following are examples. Other compiler extensions are welcome.

- **Language features:** Add `break` and `continue`, a limited form of tuples or structures, or another small language construct.
- **Program analyses:** Implement liveness, reaching definitions, or constant analysis, with a way to inspect or use the results.
- **Optimizations:** Implement dead-code elimination, common-subexpression elimination, a restricted form of LICM, or another transformation with explicit application conditions.
- **Backend features:** Implement instruction selection or another backend component for a small, specified input subset.

Related features may be combined, such as an analysis and a transformation that uses its results. Keep the scope manageable; implementing a complete language feature set or backend is not expected.

3. Implementation

Modify the files needed by your extension and include any necessary build changes.

For LLVM IR optimization projects, the existing entry point is:

```
val run_optimization : Llvm.llmodule -> unit
```

The `--opt` option invokes this function in `bin/passes.ml`. You may implement the transformation there or call additional modules. Other projects may use appropriate entry points elsewhere in the compiler.

Existing libraries and LLVM passes may support preprocessing, implementation, or evaluation. Explain their roles. Invoking an existing pass alone does not constitute the core implementation contribution.

4. Proposal and Progress Presentation

Each student will give a **10-15 minute presentation** on an assigned date: **November 18, 23, 25, 30, or December 2**.

Present:

- The problem and proposed extension.
- A small example showing the expected behavior or result.
- The supported scope, core goal, and design.
- Current progress, remaining work, and planned evaluation.

The implementation does not need to be complete at presentation time. A proposal, early experiments, or a partial implementation is sufficient.

5. Correctness and Evaluation

State what your extension promises and explain why your implementation meets that promise.

- **Language features:** Specify the semantics and explain the relevant typing and code-generation rules.
- **Analyses:** Define the reported facts and justify the analysis rules and approximations.
- **Optimizations:** Explain the application conditions and why the transformation preserves permitted behavior. Consider effects and undefined behavior where relevant.
- **Backend features:** Explain how the supported source or IR operations are implemented by the target instructions.

Support your explanation with tests. Include normal cases, boundaries, feature interactions, and relevant cases that must be rejected or left unchanged. A machine-checked proof is not required.

Choose evaluation measures appropriate to your goals, such as supported constructs, analysis precision, transformed instructions, code size, or execution time. **Runtime speedup is not required for every project.** Explain the baseline and limitations, and keep other compiler settings equal when comparing an optimization.

LLVM verifier success checks structural validity. Additional reasoning and testing are needed to support semantic correctness. Alive2 may provide additional evidence for supported LLVM IR transformations.

6. Final Submission

Push the following to **your own GitHub repository by December 13, 2026, at 23:59 (KST)**:

- **Source code:** Modified and added files, including build configuration.
- **Tests and evaluation:** Inputs, expected results, scripts, and results discussed in your report.
- **README:** Dependencies, build and execution instructions, and commands to reproduce tests and evaluation.
- **A short PDF report:** Problem and scope, specification, design and implementation, correctness argument, evaluation, limitations, and references.

Clearly identify your contribution and distinguish completed work from remaining work.

7. Assessment

Assessment considers technical substance, implementation quality, correctness reasoning, tests and evaluation, reproducibility, and presentation and report clarity.

Projects are evaluated against their stated goals and supported scope. A focused implementation with clear reasoning and convincing evaluation is sufficient; a larger feature count is not itself a goal.